

Architecting a Flash-Based Storage System for Low-Cost Inference of Extreme-Scale DNNs

Yunho Jin^{ID}, Shine Kim, Tae Jun Ham^{ID}, and Jae W. Lee^{ID}, *Senior Member, IEEE*

Abstract—The size of deep neural network (DNN) models has been exploding rapidly, demanding a colossal amount of memory capacity. For example, Google has recently scaled its Switch Transformer to have a parameter size of up to 6.4 TB. However, today's HBM DRAM-based memory system for GPUs and DNN accelerators is suboptimal for these extreme-scale DNNs as it fails to provide enough capacity while its massive bandwidth is poorly utilized. Thus, we propose *Leviathan*, a DNN inference accelerator, which integrates a cost-effective *flash-based* storage system, instead. We carefully architect the storage system to provide enough memory bandwidth while preventing performance drop caused by read disturbance errors. Our evaluation of *Leviathan* demonstrates an 8.3× throughput gain compared to the iso-FLOPS DNN accelerator with conventional SSDs and up to 19.5× higher memory cost-efficiency than the HBM-based DNN accelerator.

Index Terms—DNN inference, deep neural networks (DNNs), hardware accelerator, solid-state drive (SSD), storage device

1 INTRODUCTION

DEEP Neural Networks (DNNs) have become more than just cutting-edge technology. They are now driving engines of various applications in fields such as Natural Language processing (NLP), computer vision, recommender systems and many more. Until recently, research for such engines focused solely on increasing the computation demand to achieve higher accuracy. However, state-of-the-art models demand colossal memory capacity as they continuously scale their sizes. This is especially true for NLP models [1], [2] which are used for applications such as language translation [3], [4], text generation [2], [5], summarization [6], [7], and sentiment analysis [1].

This explosive increase in DNN model sizes is one of the most important trends for achieving high accuracy since the introduction of Transformers. Vaswani et al. proposed Transformers [8] in 2017 to solve NLP tasks. It allows DNN models to distinguish the importance between parts of inputs, bringing the artificial intelligence one step closer to how humans process sensory inputs. Since then, a myriad of Transformer-based models have emerged, such as BERT [1] and GPT [2]. These models demonstrate that

Transformer-based language models trained with a large corpus can produce a single model that can excel in diverse NLP tasks after few steps of fine-tuning. Specifically, GPT-3 [2], an expanded version of GPT-2, has outperformed its predecessors in various NLP-tasks with its massive size.

GPT-3 was not the first attempt to enlarge model capacity. Fig. 1 shows that model capacity has been steadily increasing; by more than three orders of magnitude over the last three years. For example, Switch Transformer from Google contains about 6.4 TB of parameters. One intuitive way to secure more capacity is to increase the number of sockets (i.e., scaling out). Taking GPT-3 as an example, one needs at least five NVIDIA's A100 GPUs, each having 80 GB of memory to secure 400 GB memory space and house the model. However, such a use of multiple devices incurs a high hardware cost. This is because high bandwidth memory (HBM) in GPU is very expensive for their high performance; however, this is unsolicited for those extreme-scale DNNs as their bandwidth requirements are often much smaller due to increased data reuse [9], [10]. Thus, we expand our view to a more efficient memory medium to store these models.

NAND-based flash memory is an attractive alternative to the HBM-based memory system for such extreme-scale DNN models as it provides a large memory capacity at a low cost [11], [12]. Recently, Behemoth [9] has proposed a novel flash-centric accelerator with Flash Memory System (FMS) to train the extreme-scale models without performance degradation compared to conventional HBM-based training accelerator. In particular, FMS exploits the characteristics of DNN training workloads to boost storage read/write throughput to tens of GB/s and extend the endurance of NAND significantly. However, despite this success, DNN inference based on NAND flash poses additional challenges that are yet to be addressed. First, the FMS exhibits far lower bandwidth than not only HBM but also the required bandwidth for inference of extreme-scale DNNs. Second, as DNN inference shows highly read-intensive,

- Yunho Jin is with the Harvard University, 02138 Cambridge, U.K. E-mail: yjin@g.harvard.edu.
- Shine Kim is with the Solution Product & Development, Memory Business, Samsung Electronics, Gyeonggi-do 18448, Republic of Korea. E-mail: s97.kim@samsung.com.
- Tae Jun Ham and Jae W. Lee are with the Seoul National University, Gwanak-gu, Seoul 08826, Republic of Korea. E-mail: {taejunham, jaewlee}@snu.ac.kr.

Manuscript received 16 January 2022; revised 25 June 2022; accepted 20 August 2022. Date of publication 28 September 2022; date of current version 11 November 2022.

This work was supported by the National Research Foundation of Korea (NRF) funded by the Korea government (MSIT) under Grant NRF-2020R1A2C3010663.

(Corresponding author: Jae W. Lee.)

Digital Object Identifier no. 10.1109/TC.2022.3209920

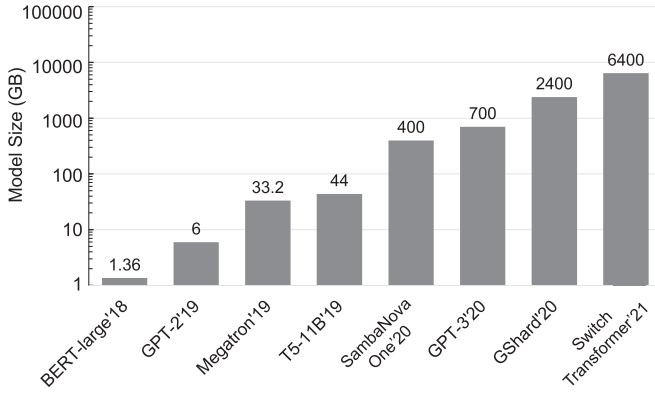


Fig. 1. Trends of model size scaling with large NLP models.

write-once-read-many (WORM) characteristic, a chronic problem in NAND flash known as *read-disturbance* (RD) [13] is exacerbated. RD-induced errors are typically addressed with read reclaim (RR) [13] and prior solutions are designed for a generic case of randomly distributed reads across blocks. This is not the case for DNN inference, where memory reads are uniformly distributed across blocks. Thus, every block reaches the threshold of RD-induced errors and calls RRs simultaneously. These bursty RRs cause a significant drop in flash read throughput.

To address these challenges, we present *Leviathan*, the first DNN inference accelerator utilizing NAND flash as main memory. The key innovation of Leviathan is realized by *Leviathan Flash Storage (LFS)*, a storage architecture specialized for serving extreme-scale DNNs. To boost the read throughput, we perform vertical optimization from the host to the storage system by leveraging the workload characteristics of DNN inference. Specifically, Leviathan exploits the read-intensive nature and static data-access pattern to facilitate flash optimizations such as cache-read and multi-plane operations for user I/O. Leviathan also resolves RD-induced performance degradation by effectively distributing RR operations over time in the course of multiple DNN inference requests. By doing so, it smooths out the peak bandwidth demanded from RR operations and minimizes interference with user I/Os for computation.

To summarize, our contributions are listed as follows.

- We analyze the workload characteristics of emerging extreme-scale DNNs to identify challenges for architecting a NAND flash-based storage system for serving those models.
- We present Leviathan, the first DNN inference accelerator replacing the high-bandwidth, low-capacity HBM with low-bandwidth, high-capacity NAND flash to hold terabyte-scale weights in a single node. We also propose techniques to boost its read bandwidth to the point where it can satisfy the bandwidth requirement of those models.
- We effectively prevent substantial throughput degradation caused by read disturbance on Leviathan Flash Storage (LFS).
- We evaluate Leviathan to demonstrate $8.29\times$ and $2.45\times$ higher inference throughput than the iso-FLOPS accelerator based on the conventional flash SSD and FMS [9], respectively. Leviathan also

A: Activation W: Weight $\rightarrow$: Forward path

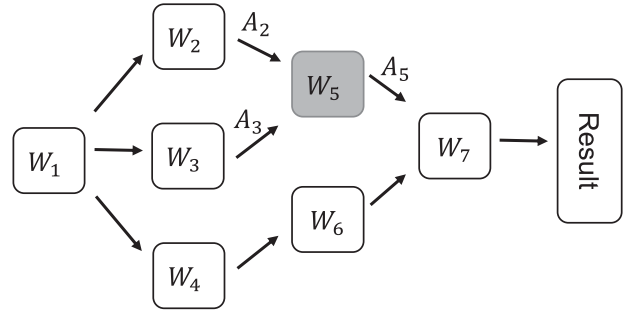


Fig. 2. DNN inference process and dataflow.

achieves up to $19.5\times$ higher memory cost-efficiency than the HBM-based DNN inference system.

2 BACKGROUND AND MOTIVATION

2.1 DNN Inference

To inference a DNN model is to exploit a trained model to carry out certain tasks it was trained for. It does so by multiplying learned weights and input data which is being propagated through the model in the form of activations. One interesting aspect of this process is that the dataflow strictly adheres to a predefined path. Therefore, it is easy to sequentialize data accesses. Also, the task features a WORM (write-once-read-many) access pattern. It reads many times from weight data that is written once. This is especially helpful in flash memories where a typical read is over $10\times$ faster than write. Leviathan exploits these properties to boost the LFS bandwidth.

Fig. 2 depicts how data flows during DNN model inference. Taking W_5 of Layer 5 as an example, it receives activations A_1 and A_2 from the two preceding layers Layer 2 and Layer 3. Layer 5 multiplies these activations with its weights W_5 to produce the activation tensor A_5 , which is in turn passed onto W_7 of Layer 7. The same process is repeated from W_1 of Layer 1 to W_7 of Layer 7, which produces the result. As such, each layer of a inference request reads the weight data once and deletes activation data once the following layers have consumed it.

2.2 Extreme-Scale Transformer-Based Models

Due to their outstanding performance, Transformers [8] are widely used in domains including NLP [1], computer vision [14], and recommendation [15]. The key primitive of Transformer is what is known as *attention* mechanism illustrated in Fig. 3a. This layer lets the model know which part of the input sequence it should attend to the most. They are stacked together to create Transformer. Some Transformer-based models are stacked with either one of the two blocks: (1) encoders [1] or (2) decoders [2] but all of them have the same architecture and the same dataflow. When used together [8], [16], additional attention is added to the decoders and the data flows as in Fig. 3b and, if not, only one of the two stacks exists.

A recent trend of the Transformer-based models is to scale their sizes to improve the model accuracy [2], [16]. This is done by widening the layer dimensions and/or stacking more encoders/decoders. Naturally, the enlarged

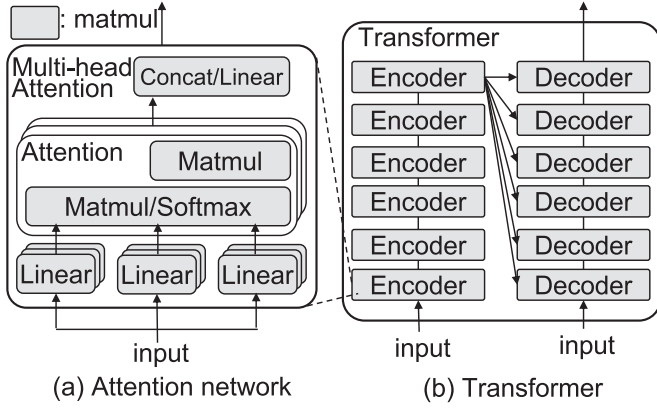


Fig. 3. Transformer model architecture. (a) Attention network and (b) Transformer.

models consume more memory capacity. For example, GPT-3 with 175 billion parameters requires 350 GB of capacity just to hold the model parameters even if we store each parameter in FP16. To address this *memory capacity wall*, we need an efficient architectural solution to continue model scaling.

2.3 Flash Memory Systems

To overcome the memory capacity wall caused by the gigantic size of modern Transformer-based NLP models, leveraging high-capacity NAND-based storage has been attracting attention [9], [10], [11]. However, replacing HBM of DNN accelerator with NAND-based storage requires dramatic improvements of the NAND-based storage in terms of throughput and endurance. Recently, Behemoth [9] has proposed a flash memory system (FMS) for extreme-scale DNN training. The FMS improves the performance by more than 3 times and endurance by more than 40 times compared to a conventional NAND-based SSD [17]. And thus, the FMS replaces the high-cost, low-capacity HBM of training accelerators with low-cost, high-capacity NAND without degrading the training throughput.

Technically, the bandwidth of a flash-based storage can be improved by utilizing a large number of NAND chips and channels as well as high-speed NAND channel interfaces. However, to fully utilize the high peak hardware bandwidth of the NAND-based storage device, the SSD firmware bottleneck running on a general-purpose processor has to be eliminated [18]. As a DNN specific storage system, FMS of Behemoth outperforms conventional NAND-based SSDs due to the fully hardware automated data path.

FMS leverages the workload characteristic of DNN to classify the training data into separated streams according to the data types. The multi-stream feature of FMS transforms all write I/Os for DNN training into strict append-only sequential I/Os. This sequential I/Os greatly simplify or even obviate the need for complicated garbage collection, wear leveling, and persistency support as performed by the modern SSD firmware. And thus, as shown in Fig. 4, the lightweight storage data path of FMS is implemented in hardware and supports over 50 GB/s of bandwidth without the SSD firmware bottleneck. Eventually, performance of

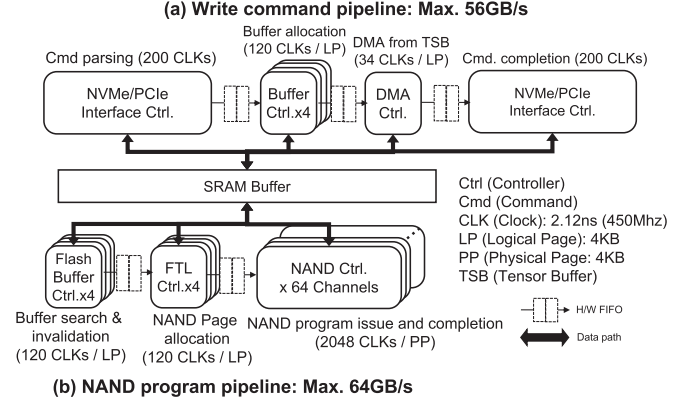


Fig. 4. Automated data path of FMS (reproduced from [9]).

FMS scales with as number of NAND chips per a flash channel, number of flash channels, NAND channel speed and controller clock speed. In addition, FMS drastically extends the endurance of NAND by leveraging tradeoffs between retention time and endurance (P/E cycles) of NAND flash memory. Although FMS provides scalable performance and increased endurance for DNN training workload, several technical challenges remain yet to be addressed for *inference* of those models.

2.4 Challenges for Extreme-Scale Model Inference

Modern extreme-scale DNN models require hundreds of gigabytes of memory space, even if each parameter is represented in FP16 format because of its gigantic size. For example, GPT-3 with 175 billion parameters requires at least 350 GB of memory space to hold only its parameters.

Insufficient Memory Capacity and Bandwidth Underutilization for HBM Conventional DNN inference platforms, such as NVIDIA GPUs and Google TPUs, adopt HBM DRAM for main memory, which offers limited storage capacity. For example, a single TPU chip, as well as a single V100 GPU only has (a maximum of) 32 GB memory space. In order to performance inference of GPT-3 on these platforms, a minimum of 66 devices are required just to hold the model parameters. In terms of computation, this is not a significant problem since training an extreme-scale model like GPT-3 in a reasonable time frame requires a very large number of FLOPS, which often exceed that of the 66 GPUs or TPU chips. However, the main waste here is that these existing platforms come with an unnecessarily expensive memory system, which is an absolute overkill for the large-scale NLP model training.

Large-scale, Transformer-based models are composed mostly of stacks of fully connected (FC) layers, which are implemented by matrix multiplication. To scale the model the matrix size in these layers continue to increase. As the matrix dimension increases, the required bandwidth per FLOPS tends to decrease due to a higher degree of data reuse. Specifically, matrix multiplication of a $m \times n$ matrix and a $n \times k$ matrix requires total mnk multiply-accumulate (MAC) operations and $mn + nk + mk$ data communications from/to memory. Enlarging each dimensions by a factor of two, the same multiplication now requires $8mnk$ MAC

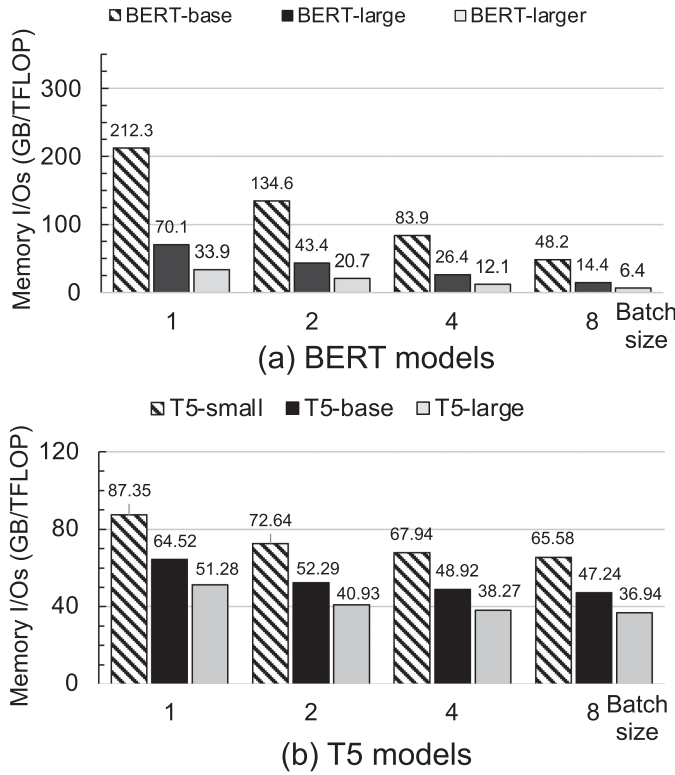


Fig. 5. Trends of memory bandwidth requirements for DNN inference as model and batch size increase.

operations and $4mn + 4nk + 4mk$ data communication. That is, computational increase is a cubic function of matrix dimension, whereas data transfer is only a quadratic function. For example, a feed forward layer in GPT-3 has two matrices with dimensions $m = 2048 \times 1, n = 12288, k = 49152$. This layer requires 2.473 TFLOP for its matrix multiplication when computing a sequence with 2048 tokens and 1.46 GB of memory transfer when using FP16 datatype. A single TPU chip has computation power of 105 TFLOPS and computes the matrix multiplication in 0.023 seconds. The HBM memory adopted by TPU v3 devices can sustain 600 GB/s aggregate bandwidth and is able to transfer 13.8 GB during this period. Thus, the memory subsystem can deliver almost an order of magnitude higher throughput than what is actually needed, to show a gross bandwidth underutilization problem. Thus, we aim to reduce the gap between the arithmetic intensity of the models and the compute-to-memory/storage bandwidth ratio of accelerators.

Fig. 5 confirms by real measurements this observation of lower relative bandwidth demands as we scale the model and batch size. We use NVIDIA GPU A100 with 40 GB

HBM [19] and models on PyTorch [20] for these experiments. We employ BERT-base [1] and T5 [21] for their widespread use. BERT-larger model is extended from BERT-large as much as BERT-large is extended from BERT but until it does not exceed the 40 GB capacity limit. The sizes of T5 models are scaled in the same way. The detailed model configurations are listed in Table 1. As shown in the figure, the required memory traffic per unit computation (TFLOPS) for the Transformer-based NLP models steadily decreases as the number of parameters or batch size increases. For instance, in the batch size one, BERT-base model performs an average 1.13 TFLOPS of computation at 239 GB/s of HBM memory throughput (i.e., 212.3 GB per TFLOP) during each training iteration. In contrast, BERT-larger model incurs less memory traffic per computation (i.e., 33.9 GB per TFLOP = 389 GB/s / 8.54 TFLOPS), which is an order of magnitude smaller than the BERT-base model due to its higher degree of data reuse with an increased model size. T5 also demonstrates similar bandwidth scaling trends.

This analysis of bandwidth underutilization indicates that HBM is not an ideal choice for this workload. A similar argument applies for NVIDIA A100 GPU, which features some 312 TFLOPS with an HBM memory system of 1555 GB/s aggregate memory bandwidth. For example, to infer GPT-3 with 175 billion parameters (i.e., 350 GB) within 1 second, the storage device only needs to provide the inference accelerator with 350 GB/s read bandwidth.

Our Work. Observing this significant memory bandwidth underutilization and insufficient memory capacity for HBM, a NAND-based storage system can be a viable alternative for the extreme-scale DNN workloads. In particular, a recent work proposes a flash memory system (FMS) to enable data parallel training of the extreme-scale DNNs [9]. Although the FMS effectively replaces the high-cost HBM DRAM of DNN training accelerators with low-cost NAND flash memory using fully hardware automated storage data path, several technical barriers need to be addressed to apply FMS to inference workload.

First, a NAND-based storage for the extreme-scale DNN inference should be able to deliver a read bandwidth of hundreds of GB/s. Although hundreds of GB/s read throughput is easy to provide with HBMs, it is a challenging target for NAND-based storage. We need to further improve the read performance of FMS (i.e., 50 GB/s per a storage node) to satisfy the requirements of the inference workload. The second challenge is reliability. DNN inference poses unique challenges for NAND flash memories. Its WORM access pattern exacerbates read disturbance (RD), an inherent reliability problem in NAND flash. This arises from the reference voltage applied to neighboring wordlines (WLs) of the target

TABLE 1
Transformer-Based NLP Model Configurations for Evaluating Memory Bandwidth Requirements

Config	Bert [1]			T5 [21]		
	Base	Large	Larger	Small	Base	Large
Layers	12	24	36	6	12	24
Hidden size	768	1024	1280	512	768	1024
Atten heads	12	16	20	8	12	16
Model size (Parameters)	110 M	340 M	750 M	60 M	220 M	770 M

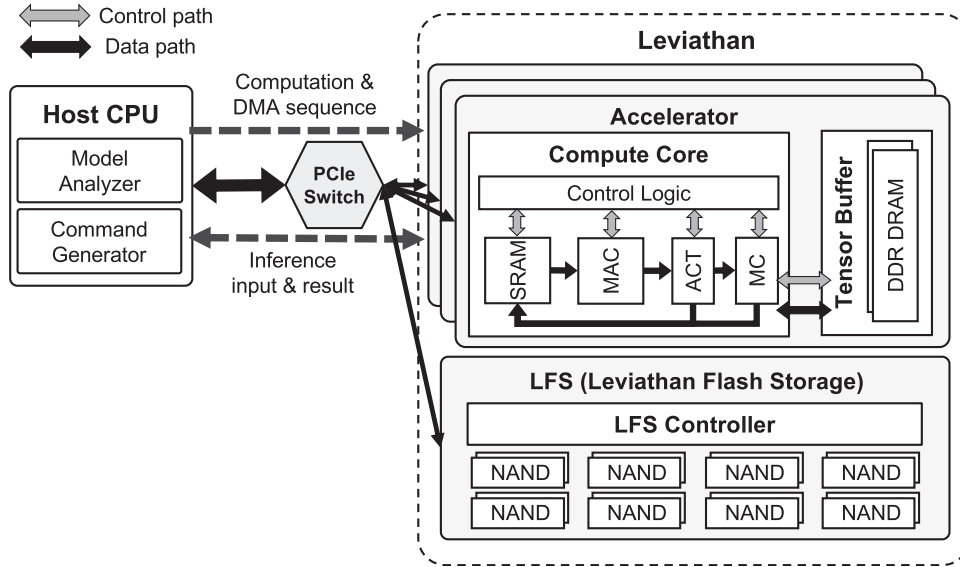


Fig. 6. Overall system architecture of Leviathan.

WL, thus soft-programming the cells in them. The WORM access pattern of DNN inference intensifies the stress on NAND cells. This stress can lead to un-correctable ECC errors in reads from the affected NAND pages if we leave them unattended [22]. Specifically, DNN inference reads each page exactly once and causes every block to experience the same number of reads. In other words, all blocks reach the RD read count threshold simultaneously. Read-reclaim (RR) is a popular technique to mitigate RD-induced reliability issues. However, conventional RR algorithms based on read count of each block [22], [23] with an assumption of non-uniform, intermittent occurrences of RD errors. They show deteriorating performance when RD-induced RR operations fire simultaneously. Therefore, the performance degradation caused by the dense RR operations must also be addressed to utilize NAND-based storage devices for inference workloads.

3 LEVIATHAN DESIGN

Leviathan is a specialized DNN inference accelerator designed for inference of extreme-scale models by addressing the bandwidth and reliability challenges in the NAND flash storage. Initially, the host CPU prepares the requested DNN model [20] and generates command sequences for both computation and DMA transfers. Then it loads the commands to the command stream in Leviathan Flash Storage (LFS) to guide model execution.

The commands are distributed to each accelerator so that they inference the model in a model-parallel manner. For example, when three accelerators are attached to a single LFS, a fully connected layer in the form of $3 M \times N$ matrix is partitioned into three $M \times N$ matrices and distributed to each accelerator. The results are accumulated in the last (third) accelerator and distributed to the Tensor Buffers in the other ones.

3.1 Preparing Model Execution on Leviathan

Model Inspection. A user defines a DNN model using a popular framework such as PyTorch [20] or TensorFlow [24]. In

the model inspection step, the model analyzer in the host traverses through a Python version of the model description and collects information including each layer's order, arguments used in the layer's operation, and input/output tensors to use. This step also conducts partitioning of a model layer such that all accelerators have balanced loads for the layer.

Command Generation. The command generator uses the collected information to generate two types of command sequences: computation command sequence and direct memory access (DMA) command sequence. The DMA command sequence orchestrates data transfers between Tensor Buffer and NAND flash devices. A DMA command includes fields such as the direction of the transfer (read/write), logical block address (LBA) of the NAND device and Tensor Buffer address. Likewise, the computation command sequence specifies the computation operations to perform on Compute Core. A computation command includes fields about the type of the layer (e.g., Fully-connected (FC), Convolution (Conv)), and the address in Tensor Buffer where the layer's input and output tensors will be stored. Both command sequences are transferred to LFS.

3.2 Hardware Components of Leviathan

The key component in Leviathan is LFS which feeds model weights to multiple accelerators in time as depicted in Fig. 6. An accelerator has Compute Core and Tensor Buffer and is connected to LFS via PCIe switch. The host delivers command sequences and inference input (i.e., sentence) to Leviathan, which in turn performs inference on the input and reports the result back to the host. We focus on the accelerator architecture in this section and elaborate more on LFS on Section 4.

Compute Core. Compute Core in its abstract form takes a similar architecture to popular commercial/academic DNN accelerators [25] as in Fig. 6. This accelerator is designed solely to accelerate DNN inference via fast matrix multiplication of weights and activations. It consists of a large 2D array of processing elements (PEs), where each PE performs

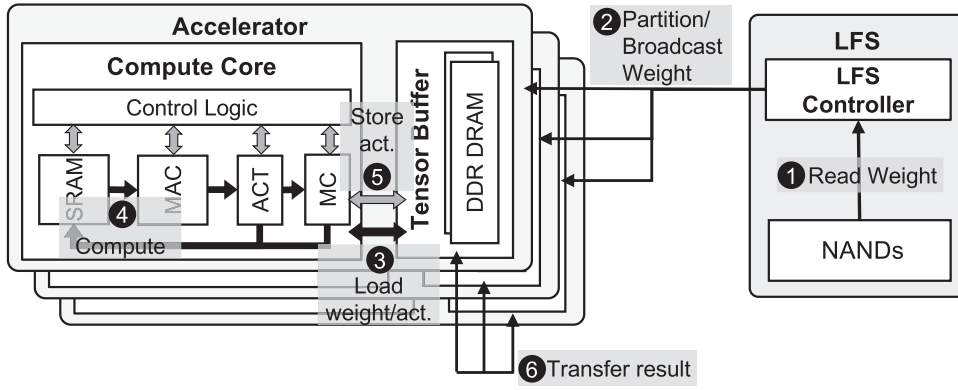


Fig. 7. Example walk-through of operations in Leviathan.

a single multiply-accumulate (MAC) operation every clock cycle. We assume weight a stationary dataflow architecture [26], where weights are directly loaded from Tensor Buffer and kept in the local registers inside the PE. However, there is no limitation for Leviathan to adopt other styles of architectures with different dataflows. Compute Core first loads the required weights and activations from Tensor Buffer through its memory controller (MC) to its SRAM. The data is passed to its MAC array for matrix multiplication at every cycle while processing a layer and the results are accumulated at the end of the layer. In case a layer in the model architecture is followed by an activation function, the result of the MAC array is passed onto the activation engine (ACT). The final result is transferred back to Tensor Buffer through MC.

Control Logic. Control Logic orchestrates the commands among the other components in Compute Core. It first decodes the commands received from the host through the PCIe bus. If all dependencies of a command are satisfied, Control Logic dispatches the command to the appropriate receiver. For example, if the command is a computation command, it is dispatched to SRAM, MAC array or ACT engine. If the command is a DMA command, it is dispatched to MC for communication between Tensor Buffer and Compute Core and between Tensor Buffer and LFS. This is a very simple logic, which sequences the commands in order.

Tensor Buffer. Tensor Buffers are placed in local DDR DRAM to serve as a staging area between Compute Core and LFS. The primary role of this buffer is to smooth the traffic between LFS and Compute Core. Tensor Buffer also serves as a communication buffer between different accelerators. Typically, a layer of an extreme-scale DNN model requires a huge amount of computation resources, and it is infeasible to be handled by a single accelerator. Thus, a layer is sharded and distributed to multiple accelerators. A shard of the layer is processed by the Compute Core in the designated accelerator and the partial result is stored in its Tensor Buffer. The sharding scheme is specified in the command sequences generated by the host and LFS feeds each accelerator with the appropriate portion of the weights following the sequences. If a following layer requires partial results from the previous layer or the results of the previous layer needs to be accumulated, the partial results are transferred from Tensor Buffer of one accelerator to the target accelerator's Tensor Buffer.

3.3 Example Execution Walk-Through

Fig. 7 illustrates how DNN inference is processed on Leviathan. Computation in Compute Core is pipelined with data transfers from LFS, Tensor Buffer, and SRAM. Data transfer is completely hidden with the help of high LFS bandwidth. Inference of a layer starts with reading its weight tensors from NAND flash **1**. The weights are partitioned and distributed to Tensor Buffers in appropriate accelerators following the DMA sequence created by the host **2**. In contrast, Tensor Buffer already houses activation tensors either as a result from the previous layer processed by the local accelerator or delivered by different accelerators. The weight and activation tensors are transferred to on-chip SRAM **3**. MAC consumes the tensors for matrix multiplication and stores the partial result to SRAM **4**. The final result is stored in Tensor Buffer **5**. It is used in the next layer by the local accelerator or transferred to a different one if needed **6**. An inference task repeats from **1** to **6** for all layers to produce the result (e.g., a generated sentence for GPT-3) and transfers it to the host.

3.4 DNN Model Coverage

Leviathan targets inference workloads for extreme-scale models whose memory bandwidth requirement does not exceed the sustainable bandwidth of LFS. Suitability for other models can be determined by analyzing their arithmetic intensity. Our model analyzer can compare the arithmetic intensity of a model with Leviathan's compute-to-bandwidth ratio (i.e., FLOPS / GB/s) to check whether the model is provided with enough bandwidth from LFS [27].

The key enabler of LFS as a storage medium of tensors is a higher degree of data reuse resulting from long sequence length. State-of-the-art models that are capable of handling such long sequences have a large sequence length. Small-sized models exhibit much lower arithmetic intensity, hence not being the primary target of Leviathan. For example, small NLP models [1] have limited sequence length (e.g., 512). This is much shorter than 2048 for GPT-3. Conventional vision models such as ResNet heavily utilize convolution layers. Converting convolutions into matrix multiplications through convolution lowering results in dimensions that are much smaller than those of FC layers. Taking ResNet-50 as an example, its largest convolutional layer has weights in 3 dimensions: $7 \times 7 \times 64$. Convolution lowering reshapes this into a 2-dimensional 7×448 matrix. Compared to 2048×12288 for FC layers in GPT-3, 7×448 for convolution layers demonstrates

much less data reuse of weights. Hence, these models are not suitable to be deployed with Leviathan. This issue can be mitigated to a certain extent by increasing the batch size, which in turn increases the degree of reuse for weight parameters.

To service accelerators to LFS data transfers without stalls, LFS should not only boost internal read bandwidth of NAND-based storage but also employ wide PCIe lanes since LFS is connected to multiple accelerators via PCIe as shown in the Fig. 6. According to our analysis, 32-lane PCIe Gen5 interconnect with dual ports is sufficient to serve the traffic over the PCIe channels. Note that each accelerator and a host system do not have to be equipped with such wide interface as LFS does.

4 LEVIATHAN FLASH STORAGE (LFS)

The WORM (Write Once Read Many) access pattern both provides opportunities and poses challenges when architecting a flash-based storage system. A highly read-intensive access pattern allows the storage system to focus on accelerating reads, which is faster than writes. However, this skewed pattern with frequent reads can exacerbate wear-outs of NAND cells and in turn increase the frequency of RD-induced errors. LFS effectively addresses these challenges to provide hundreds of GB/s read bandwidth without deterioration from RD-induced errors.

4.1 Boosting Read Bandwidth

NAND product with over 2.0 GB/s (i.e., 2.0 Gbps $\times$ 8 DDR bus) [28] and controller with automated read path [29] have been introduced to support the high bandwidth in various applications. But this still falls short in providing hundreds of GB/s required for DNN inference. Therefore, LFS augments the high bandwidth NAND and read path hardware automation with additional optimizations.

Specifically, LFS leverages cache-read and multi-plane operations, which are standard features included in Open NAND Flash Interface (ONFI) specifications [30]. Thus, most commercial NAND flash manufacturers support these features in their NAND flash products for high-bandwidth read operations. However, the actual use of these performance boosting features for user-level read requests (not internal reads) faces technical issues. We resolve these technical barriers by leveraging the workload characteristics of DNN inference to boost the read performance of LFS, eliciting the full potential of NAND.

Cache-read Operation. Cache-read overlaps NAND read and channel transfers by adding an additional buffer named data register. This is enabled by placing a cache register between NAND blocks and flash controller. Figs. 8a and 8b show page read without and with cache-read operation, respectively. Cache-read operation successfully hides the channel transfer time with the page sensing time. Most modern NAND chips employ multiple registers and additional commands to enable cache-read [28].

In spite of the available hardware logic, most commodity SSDs do not actively use the cache-read operation as a method to read user data [29]. This is because the cache-read tends to delay the current read I/O if there is no other pending read for the same NAND die. As a result, it can potentially increase the read latency [31] as the continuous use of the same I/O die is not guaranteed. Thus, many of

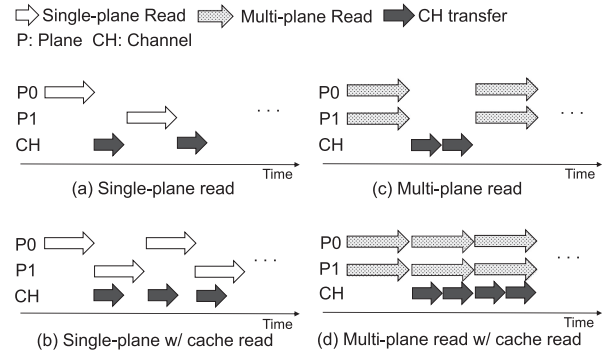


Fig. 8. A NAND chip read pipeline diagram according to read methods. Cache-read pipelines page reads and channel transfers. Multi-plane operation slightly increases the read latency but allows the device to utilize its internal parallelism.

the conventional SSDs do not enable cache-read to minimize latency for more common I/Os (e.g., small-sized low queue depth (QD) reads) than large-sized high-QD reads.

In contrast, Leviathan exploits the fact that the DNN accelerator only issues sequential, large read requests with high QD to its LFS (e.g., 2 MB $\times$ 12 QD). In fact, it is guaranteed that the DNN accelerator always requests more than two pending read I/Os to a particular NAND die at any given time. This is in contrast to other workloads with intermittent read requests (e.g., 2 MB $\times$ 1 QD). For these workloads, SSDs cannot leverage cache-read operation to pipeline two requests since there is only one request in the queue. Therefore, LFS does not incur additional read latency from the use of cache-read operations. The active use of cache-read operations, coupled with multi-plane read operation based on plane/channel/way (PCW) striping [32] explained in the next subsection, dramatically increases the effective bandwidth.

Multi-plane Operation. Multi-plane operation is a widely known technique that reads/writes multiple pages from different NAND planes in parallel, as depicted in Fig. 8c. This technique significantly increases not only the write bandwidth of an SSD but also its internal garbage collection (GC) efficiency due to parallelized plane operation [33].

Unfortunately, as of now, it is arduous to apply this useful technique to user reads. This is because most commercial SSDs utilize channel/way/plane (CWP) striping to maximize channel utilization [29], as shown in Fig. 9a. Such a striping scheme is beneficial since a small read request (i.e., 64 pages) can fully utilize all channels. However, the striping scheme makes multi-plane operation virtually impossible because the same pages across different planes are 512 pages apart. In this case, a multi-plane operation can only be issued for a sequence of reads with a specific stride (i.e., 512).

LFS utilizes plane/channel/way (PCW) striping, which is better suited for multi-plane read operations, as shown in Fig. 9b. In this case, adjacent pages are placed on the same plane across different planes, and thus it is possible to fully leverage multi-plane operations. The caveat of this approach is that a small read may not be able to utilize all channels. For example, a read request for 64 pages chunk can only utilize 16 channels out of the available 64 channels since four nearby pages are placed on the locations sharing the same channel. Fortunately, this is not an issue in Leviathan. LFS in Leviathan only utilizes large, sequential access

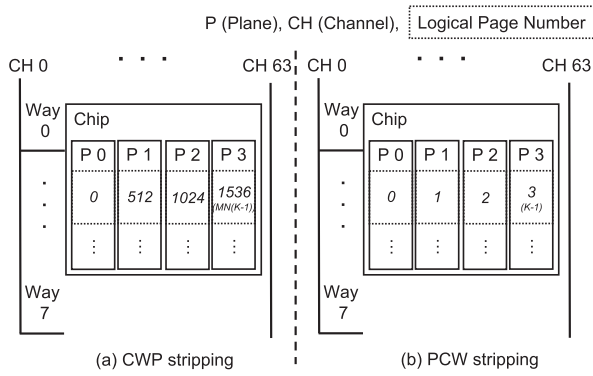


Fig. 9. Logical page mapping with two striping schemes ((a) CWP, (b) PCW) after sequential write on an SSD with 64 channels ($M = 64$), 8 ways ($N = 8$), and 4 planes ($K = 4$).

patterns, which allows all channels (and ways within channels) to be utilized regardless of the striping schemes. As a result, LFS can effectively utilize multi-plane operations with the PCW striping scheme with no penalty.

4.2 Preventing RD-Induced Performance Degradation

RD Errors in DNN Inference. Although DNN does provide opportunities for specializing LFS to increase throughput, it also poses reliability challenges. The intensive reads and their WORM pattern accelerates the RD-induced reliability problem in the flash-based memory system.

The WORM nature of DNN inference quickly fills the RD threshold. All NAND blocks in the flash memory experience almost the same degree of read stress as all weights are read once for a single inference. Therefore, every NAND block containing model weights exceeds the RD threshold *at the same time* and necessitates read reclaim (RR) operation simultaneously. Such bursty RR operations firing over a small time window severely impede the user because RR for blocks that exceeded RD threshold must be given higher priority than user read requests. In this case, RR operations significantly degrade the device's performance, thereby worsening end-to-end quality-of-service (QoS). Note that this degradation continues until every block is reclaimed. Thus, we propose a new RR smoothing technique that effectively eliminates the performance degradation induced by this bursty RR operations. **RR Timing.** The main difference between the conventional and Leviathan's RR procedures is that the former migrates *after* and the latter migrates *before* the read count reaches RD threshold [34]. The static, deterministic pattern of data accesses in DNN inference enables accurate prediction of read counts (i.e., RD stress) for all blocks. Exploiting this characteristic, Leviathan issues RR operations alongside user read requests before read count in any block exceeds the RD threshold. To put it another way, LFS gives the same priority to RR requests and inference reads, unlike the RR priority scheduling in conventional SSDs. The distribution is carefully tuned to prevent any inference performance degradation and reprogram every NAND block only once before reaching the RD threshold.

The RR timing is calculated by dividing the RD threshold associated with a program/erase (P/E) cycle, a measure of cell wear out, by pages per block ($\frac{RD_{th}}{ppb}$). For example, 768 pages per block and RD threshold of 6 M read counts at 75 K

TABLE 2
DNN Models Evaluated With Leviathan (Token Size: 2048)

Model Size	BERT/GPT-like [1], [2]			T5-like [21]		
	Act. (GB)	Weight (GB)	PFLOP	Act. (GB)	Weight (GB)	PFLOP
1×1	44	350	2.15	40	305	0.62
1×2	88	698	4.42	80	609	1.25
1×4	175	1393	8.56	160	1218	2.49
2×1	88	1395	8.56	80	1218	2.49
2×2	175	2786	17.12	160	2436	4.99
2×4	349	5569	34.21	319	4871	9.97

P/E cycles require full read reclaim of all blocks for every 7813 inference requests. The timing is set as such due to two reasons: (1) a read in the page disturbs every page in the same block and (2) every page is read exactly once during model inference. Namely, read count of a page after one inference request is equivalent to the number of pages per block. Note that NAND block allocation policy for RR is based on an existing proposal [9].

LFS Resource Reservation for RR. Though RR request in the above example is not frequent, the corresponding bandwidth must be reserved by the SSD for execution of DNN inference without interruption via effective RD resolution. Unlike the DNN inference workload that only issues read requests sequentially, a RR operation, which is a type of GC, *reads* source pages, *erases* a destination block, and *programs* into the block. This resource consumption calls for the need to pay careful attention to factors such as NAND topology, channel bandwidth, FTL scheme, and degree of hardware automation. Specifically, user read I/O to a NAND chip is blocked for a few milliseconds during the block erase operation for RR operation, thus a read prefetch buffer of appropriate size is required for a host system [11] or an SSD. Otherwise, erase suspension technique [35] can be adopted to meet the QoS requirement for DNN inference. Leviathan reserves about 1% of channel bandwidth for the RR operation as the use of low latency SLC NAND [17], automated read/write datapath [9], [29], and bandwidth boost techniques (Section 4.1) stress NAND channels as a potential bottleneck in LFS. Also, each accelerator reserves 1 GB space in its Tensor Buffers to prevent QoS degradation caused by block erase operations in RR.

5 EVALUATION

5.1 Methodology

Simulators. We use MAESTRO [36] and MQSim [37] for cycle-level modeling of DNN accelerator and SSD, respectively, and PyTorch [20]. We assume that the time constraints of GPT-3 model for inference is 1.3 s, 10× of the known time constraint for a BERT model [38]. This constraint is conservative considering that GPT-3 model is about 500× larger than the BERT model.

Workloads. We evaluate twelve workloads with Leviathan. The models represent two types of widely adopted Transformer-based models: 1) BERT/GPT-like and 2) T5-like, as listed in Table 2. They are grouped in this way such that the models in the group share an identical architecture. Specifically, BERT/GPT-like models do not have cross-

TABLE 3
Configurations for the Inference Throughput and Cost Evaluation

Accelerator Configurations				
Number of cores	12 cores (26.25 TFLOPs per core)			
	Memory Configurations			
	SSD	FMS	LFS	HBM
Buffer conf.	48 GB DDR DRAM + 1 TB NAND	384GB		
Peak Read Bandwidth	32 GB/s	108 GB/s	290 GB/s	7200 GB/s

TABLE 4
Conventional SSD, FMS, and LFS Configurations

	SSD	FMS	LFS
Storage Configuration			
Capacity	250GB × 4 SSDs (RAID 0)	1TB	1TB
NAND Configurations			
Topology	32 CHs, 1 chip/CH, 2.4GB/s/CH	128 CHs, 1 chip/CH, 2.4GB/s/CH	128 CHs, 1 chip/CH, 2.4GB/s/CH
Structure	64Gb SLC / die, 8 planes / die, 342 blocks / plane, 768 pages / block, 4KB page		
Latency	Single-plane read: 3μs, Multi-plane read: 8μs, Program: 100μs, Block erase: 5ms		
Read Disturb Threshold	1M reads / block		
FTL Configurations			
Stripe Policy	CH/Way/ Plane	CH/Way/ Plane	Plane/CH/ Way
Multi-plane Read	GC-only	GC-only	User, GC
Cache Read	X	X	O
Firmware Latency			
Read	1μs / a page	N/A	N/A
Write	1.45μs / a page	N/A	N/A

attention layer in their decoder stacks, while T5-like models do. We stacked (Depth) more encoder/decoders and expanded the dimensions in FC layers (Width) of these generic Transformer-based models for diverse configurations. They are referred to as $W \times D$ in the rest of this paper.

Baselines. We evaluated Leviathan against two systems: one system equipped with conventional SSDs and the other equipped with FMS. We also compare Leviathan with accelerators connected to sufficient HBM that eliminates memory bandwidth bottleneck. The accelerator and storage configurations are tabulated in Tables 3 and 4. Note that, for the models

exceeding the memory capacity in the tables, we varied the capacity of memory devices for an HBM-based accelerator from 384 GB to 5.76 TB and NAND-based accelerators (i.e., SSD, FMS and LFS) from 1 TB to 6 TB, respectively, to accommodate each model in evaluations by adding memory chips; the number of accelerators is set to 200 so that each evaluated configuration shows the same computation performance.

5.2 Performance of DNN Inference

We evaluate Leviathan's inference throughput and calculate the required memory cost in serving those inference queries. The batch size used for BERT/GPT-like and T5-like models are 35 and 110, respectively.

Overall Inference Throughput. Fig. 10 illustrates the throughput of the four systems. Leviathan achieves the same throughput as the HBM-based accelerator in all cases. In contrast, FMS shows 2.45× lower inference throughput than Leviathan with LFS on average. This is because LFS harnesses the full potential of NAND flash memory by using cache read and multi-plane read operations for user-level read I/Os. The accelerator with conventional SSDs achieves limited throughput because of the SSD firmware bottleneck. In 1×1 case of the BERT/GPT-like model, the presented Leviathan system utilized 282 GB/s aggregate memory system bandwidth. About 1% extra bandwidth of the 282 GB/s is reserved for RR. Since LFS can provide 290 GB/s bandwidth, the system maintains high throughput without being limited by the LFS bandwidth. The reason why the raw throughput decreases for a larger model is that it takes longer to run on the fixed hardware (in Table 4). However, the relative performance gains of LFS over FMS are more or less preserved regardless of the model size.

Memory Cost Efficiency. The difference in memory cost between Leviathan and the conventional HBM-based accelerator is shown in Fig. 11. For this calculation, we assumed HBM device cost to be \$20/GB [39], DDR4 DRAM device cost to be \$4/GB [40], and flash device cost to be \$0.81/GB. Note that the cost of flash device for LFS, using 64 Gb V-NAND based SLC, was set as 4.8 times a commercial SSD price (0.167\$/GB) which uses 512 Gb V-NAND TLC [41] considering the additional peripheral costs for 64 Gb SLC NAND. It can be seen from the figure that the cost gap between the two systems increases as we scale the model size. The maximum difference of \$25.7 M for BERT/GPT-like models and \$7.5 M for T5-like models.

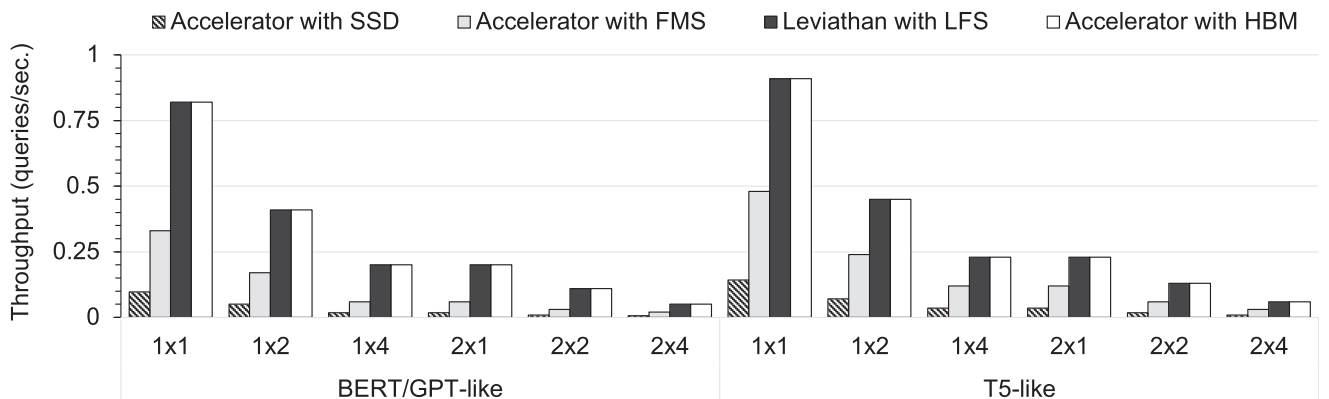


Fig. 10. Inference throughput comparison.

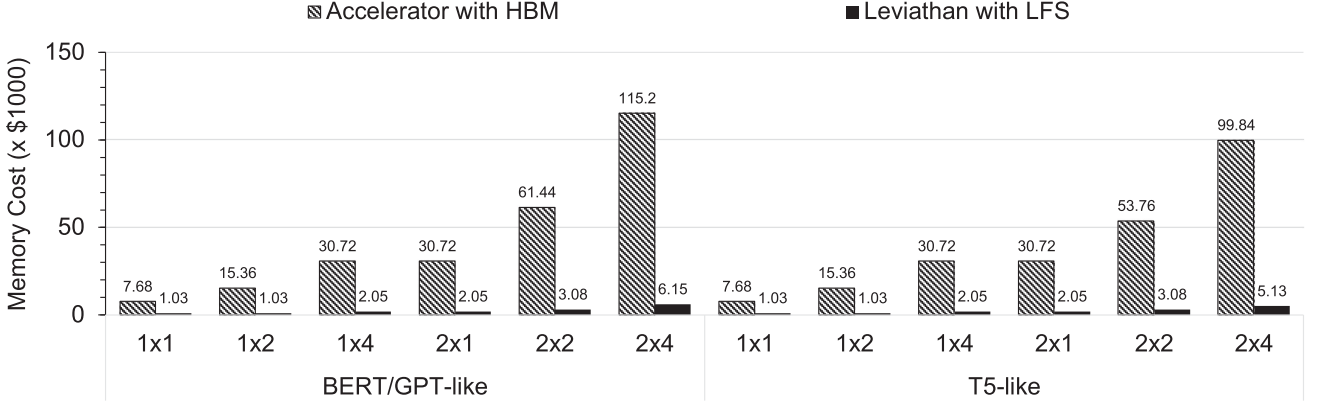


Fig. 11. Memory system cost comparison.

5.3 Bandwidth Variability During Read Reclaim

Fig. 12 depicts the storage performance for inference workloads of five NAND-based configurations. We align the start time of RR operations at n for all four configurations. Originally, the start times are different due to the different speeds in reaching RD threshold as they provide disparate read speeds for inference. When there is no RR, cache-read enhances the read bandwidth of FMS by $1.59\times$ by pipelining NAND page read time with channel DMA time. Multi-plane operations give additional throughput gains by $1.65\times$ by reading data from multiple NAND planes. By combining both optimizations, LFS achieves over 90% of NAND channel bandwidth utilization, which is in line with the numbers shown in Table 4. However, during RR, all systems but LFS show severe bandwidth deterioration (by up to 88%) from bursty RRs after n seconds for a maximum of 16 seconds. This is the time needed to reprogram all weight tensors. Other four systems maintain similar bandwidth during RR since multi-plane operation is already being carried out for RR. In contrast, LFS shows more robust performance, free from RD-induced bandwidth drop by distributing RR operations over time.

LFS builds on top of FMS to inherit the fully hardware automated datapath. However, FMS focuses on improving write performance and device lifetime for training and fails

to satisfy the read performance requirements for extreme-scale DNNs. LFS overcomes these limitations by employing read optimizations such as cache-read and multi-plane operations. Furthermore, it also optimizes the stripe-policy to enable multi-plane operations for not only garbage collections but also user read requests as shown in Table 4.

6 RELATED WORK

Heterogeneous Memory for DNN Training. Recent literature has been shifting the direction to heterogeneous memory [9], [10], [11], [12] to follow the rapid increase in capacity demand from the DNN models. Specifically, many of them append SSDs to DNN training system's memory hierarchy to enlarge the memory capacity of the system.

FlashNeuron [11] stores weight tensors that are to be used in the near future on GPU memory and offloads others to SSD, fetching them whenever GPU memory becomes available. Specifically, it records a tensors lifetime, from its creation to reuse and deletion. This information is used when it offloads tensors that are used in the distant future to SSD and keep those that are used in the near future in GPU memory. It facilitates peer-to-peer communication between GPU and SSD by opening their memory space to each other. The exposure allows data to be transferred directly between the two PCIe peripherals without CPU intervention. This work focuses on offloading and prefetching of weight tensors during training and enlarges the memory space seen by GPU. FlashNeuron resolves the memory capacity wall in training very deep neural networks and increases training throughput by scaling the batch size.

ZeRO-Infinity [10] introduces a heterogeneous memory architecture to break through the memory capacity wall of GPU. Zero-Infinity exploits a variety of types of memories and computation cores in a system (e.g., HBM, DRAM, NVMe, CPU, and GPU) to train extreme-scale DNN models without requiring model code refactoring. Zero-Infinity utilizes the commodity hardware to train up to 32 trillion parameters using data parallelism with significantly higher performance than existing model parallelism. In particular, considering the bandwidth and capacity of memory devices and computational throughput of cores, Zero-Infinity automatically deploys the extreme-scale Transformer model to numerous training hardware.

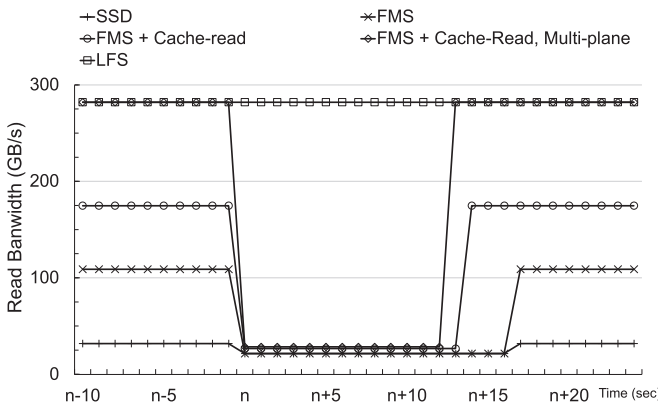


Fig. 12. Bandwidth variations during read reclaim (RR). RR is repeated every n seconds with varying n for each system. RR starting time of each storage configuration (n) is (1) SSD: 14,242 (2) FMS: 4,180, (3) FMS w/ Cache-read: 2,608, (4) FMS w/ Cache-read, Multi-plane read: 1,615, (5) LFS: 1,615 (sec).

Zhao et al. [12] focuses on advertisement systems and use batch I/O to fully utilize SSD bandwidth. They not only run conventional SSD-based storage system as a parameter server but also propose hierarchical memory architecture which exploits HBM of GPUs, host system memory and commodity SSDs for massive scale sparse DNN model training.

The aforementioned prior works use conventional SSDs and thus they do not exploit the data access patterns specific to DNN training to increase the storage bandwidth. In contrast, Behemoth [9] introduces a new domain specific SSD that utilizes the sequential data access pattern of DNN training to remove the firmware overhead from SSD write path and boost the bandwidth. Previous works focus their solutions to increase the SSD bandwidth to tens of GB/s, which is enough to satisfy DNN training but comes short in satisfying hundreds of GB/s of bandwidth required for DNN inference.

Read Disturbance. Read disturbance (RD) is an inherent reliability problem that arises when reading from NAND flash memories. Especially, conventional read-reclaim algorithms based on read count of each block [22], [23], [34] are designed for cases where RD induced errors occur randomly. These techniques display deteriorating performance when applied to storage for DNN inference where every block exceeds the read threshold simultaneously. Leviathan avoids this by distributing the RR operations evenly among read requests from DNN inference. In effect, it smooth out the peak delay caused by simultaneous RR.

In-storage Processing. In-storage processing (ISP) is a promising direction to accelerate workloads that require large memory capacity. For example, there are ISP proposals to accelerate query processing for databases [42] and machine learning [43], [44], [45]. All of these proposals focus on integrating compute accelerators into a storage device by utilizing either embedded CPUs and internal DRAM [42], [44] or custom hardware accelerators [43], [45].

In contrast, Leviathan is not an in-storage processing system as both the accelerator device and the LFS device are attached to a PCIe switch. The main focus of Leviathan is LFS, an application-specific NAND flash device architecture, and the near-storage processing solution based on LFS. It addresses the issues of not only storage performance but also failure mitigation, which is critical on a NAND flash-based DNN inference system. Leviathan also targets different applications from RecSSD [44] and DeepStore [43], which target recommender system and intelligent database query processing, respectively.

7 CONCLUSION

This article presents Leviathan, a DNN inference accelerator designed to fully accommodate extreme-scale models using cost-efficient NAND flash memory. Leveraging the static, deterministic access pattern of DNN inference, it successfully increases the flash memory bandwidth and averts detrimental cases arising from simultaneous, bursty RRs. The bandwidth is boosted using cache-read and multi-plane operations, while RD related issue is resolved by distributing RR between user read requests. With Leviathan, DNN inference can be run on a much cheaper memory system than the conventional HBM-based memory system.

ACKNOWLEDGMENTS

Yunho Jin and Shine Kim contributed equally to this work.

REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," Oct. 2018, *arXiv:1810.04805*.
- [2] T. B. Brown et al., "Language models are few-shot learners," May 2020.
- [3] A. Vaswani et al., "Tensor2Tensor for neural machine translation," Mar. 2018, *arXiv:1803.07416*.
- [4] M. Ott, S. Edunov, D. Grangier, and M. Auli, "Scaling neural machine translation," Jun. 2018, *arXiv:1806.00187*.
- [5] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, 2019.
- [6] H. Zhang, J. Xu, and J. Wang, "Pretraining-based natural language generation for text summarization," Feb. 2019, *arXiv:1902.09243*.
- [7] W. Kryściński, N. Shirish Keskar, B. McCann, C. Xiong, and R. Socher, "Neural text summarization: A critical evaluation," Aug. 2019, *arXiv:1908.08960*.
- [8] A. Vaswani et al., "Attention is all you need," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
- [9] S. Kim, Y. Jin, G. Sohn, J. Bae, T. J. Ham, and J. W. Lee, "Behemoth: A flash-centric training accelerator for extreme-scale DNNs," in *Proc. 19th USENIX Conf. File Storage Technol. Assoc.*, 2021, pp. 371–385.
- [10] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-infinity: Breaking the GPU memory wall for extreme scale deep learning," Apr. 2021, *arXiv:2104.07857*.
- [11] J. Bae et al., "FlashNeuron: SSD-enabled large-batch training of very deep neural networks," in *Proc. 19th USENIX Conf. File Storage Technol.*, 2021, pp. 387–401.
- [12] W. Zhao et al., "Distributed hierarchical GPU parameter server for massive scale deep learning ads systems," Mar. 2020, *arXiv:2003.05622*.
- [13] Y. Cai, Y. Luo, S. Ghose, and O. Mutlu, "Read disturb errors in MLC nand flash memory: Characterization, mitigation, and recovery," in *Proc. IEEE/IFIP 45th Annu. Int. Conf. Dependable Syst. Netw.*, 2015, pp. 438–449.
- [14] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition At Scale," Oct. 2020, *arXiv:2010.11929*.
- [15] F. Sun et al., "BERT4Rec: Sequential Recommendation With Bidirectional Encoder Representations From Transformer," Apr. 2019, *arXiv:1904.06690*.
- [16] Microsoft, "turing-NLGMicrosoft," 2020. [Online]. Available: <https://www.microsoft.com/en-us/research/blog/turing-nlg-a-17-billion-parameter-language-model-by-microsoft/>
- [17] Samsung, "Z-SSD SamsungSZ985," 2018. [Online]. Available: https://www.samsung.com/semiconductor/global.semi.static/Brochure_Samsung_S-ZZD_SZ985_1804.pdf
- [18] J. Zhang, M. Kwon, M. Swift, and M. Jung, "Scalable parallel flash firmware for many-core architectures," in *Proc. 18th USENIX Conf. File Storage Technol.*, 2020, pp. 121–136. [Online]. Available: <https://www.usenix.org/conference/fast20/presentation/zhang-jie>
- [19] NVIDIA, "NVIDIA A100 GPU," 2020. [Online]. Available: <https://www.nvidia.com/en-us/data-center/a100/>
- [20] pytorch.org, "PyTorch," 2021. [Online]. Available: <https://pytorch.org>
- [21] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text Transformer," Oct. 2019, *arXiv:1910.10683*.
- [22] K. Ha, J. Jeong, and J. Kim, "An integrated approach for managing read disturbs in high-density nand flash memory," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 35, no. 7, pp. 1079–1091, Jul. 2016.
- [23] B. S. Kim, J. Choi, and S. L. Min, "Design tradeoffs for SSD reliability," in *Proc. 17th USENIX Conf. File Storage Technol.*, 2019, pp. 281–294.
- [24] M. Abadi et al., "Tensorflow: Large-scale machine learning on heterogeneous distributed systems," Mar. 2016, *arXiv:1603.04467*.
- [25] N. P. Jouppi et al., "<120>," "In-datacenter performance analysis of a tensor processing unit," *SIGARCH Comput. Archit. News*, vol. 45, no. 2, pp. 1–12, Jun. 2017.
- [26] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proc. 43rd Int. Symp. Comput. Architect.*, 2016, pp. 367–379.

- [27] Y. H. Oh et al., "Layerweaver: Maximizing resource utilization of neural processing units via layer-wise scheduling," in *Proc. IEEE Int. Symp. High- Perform. Comput. Architect.*, 2021, pp. 584–597.
- [28] J. Cho et al., "30.3 A 512Gb 3b/cell 7th-generation 3D-NAND flash memory with 184mb/s write throughput and 2.0gb/s interface," in *Proc. IEEE Int. Solid-State Circuits Conf.*, 2021, pp. 426–428.
- [29] W. Cheong et al., "A flash memory controller for 15 μ s ultra-low-latency SSD using high-speed 3D NAND flash with 3 μ s read time," in *Proc. IEEE Int. Solid-State Circuits Conf.*, 2018, pp. 338–340.
- [30] Micron Inc., "Onfi: Open nand flash interface specification 5.0." [Online]. Available: https://media-www.micron.com/-/media/client/onfi/specs/onfi_5_0_gold.pdf?la=en&rev=b9d79143b14143a7a8253c1ae20b247c
- [31] J. Woo, M. Ahn, G. Lee, and J. Jeong, "D2FQ: Device-direct fair queueing for NVMe SSDs," in *Proc. 19th USENIX Conf. File Storage Technol.*, 2021, pp. 403–415.
- [32] A. Tavakkol, P. Mehrvarzy, M. Arjomand, and H. Sarbazi-Azad, "Performance evaluation of dynamic page allocation strategies in SSDs," *ACM Trans. Model. Perform. Eval. Comput. Syst.*, vol. 1, no. 2, pp. 1–33, Jun. 2016.
- [33] N. Shahidi, M. T. Kandemir, M. Arjomand, C. R. Das, M. Jung, and A. Sivasubramanian, "Exploring the potentials of parallel garbage collection in SSDs for enterprise storage systems," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2016, pp. 561–572.
- [34] C.-Y. Liu, Y. Lee, M. Jung, M. T. Kandemir, and W. Choi, "Prolonging 3D NAND SSD lifetime via read latency relaxation," in *Proc. 26th ACM Int. Conf. Architect. Support Program. Lang. Operating Syst.*, 2021, pp. 730–742.
- [35] S. Kim et al., "Practical erase suspension for modern low-latency ssds," in *Proc. USENIX Annu. Tech. Conf.*, 2019, pp. 813–820.
- [36] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of DNN dataflow: A data-centric approach," in *Proc. IEEE/ACM 52nd Annu. Int. Symp. Microarchitecture*, 2019, pp. 754–768.
- [37] A. Tavakkol, J. Gómez-Luna, M. Sadrosadati, S. Ghose, and O. Mutlu, "MQSim: A framework for enabling realistic studies of modern multi-queue SSD devices," in *Proc. 16th USENIX Conf. File Storage Technol. USENIX Assoc.*, 2018, pp. 49–66.
- [38] V. J. Reddi et al., "MLPerf inference benchmark," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Architect.*, 2020, pp. 446–459.
- [39] Fudzilla.com, "Radeon VII 16 GB HBM 2 memory cost around \$320." [Online]. Available: <https://www.fudzilla.com/news/graphics/48019-radeon-vii-16gb-hbm-2-memory-cost-around-320>
- [40] "Newegg.com." [Online]. Available: <https://www.newegg.com>
- [41] "SSD price: Samsung SSD 970 EVO." [Online]. Available: https://www.amazon.com/Samsung-970-EVO-1TB-MZ-V7E1T0BW/dp/B07BN217QG/ref1/4sr_1_2?dchild=1&keywords=970evo&qid=1600645757&sr=8-2
- [42] B. Gu et al., "Biscuit: A framework for near-data processing of big data workloads," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Architect.*, 2016, pp. 153–165.
- [43] V. S. Mailthody et al., "Deepstore: In-storage acceleration for intelligent queries," in *Proc. IEEE/ACM 52nd Annu. Int. Symp. Microarchitecture*, 2019, pp. 224–238.
- [44] M. Wilkening et al., "RecSSD: Near data processing for solid state drive based recommendation inference," in *Proc. 26th ACM Int. Conf. Architect. Support Programm. Lang. Operating Syst.*, 2021, pp. 717–729.
- [45] M. Kwon, D. Gouk, S. Lee, and M. Jung, "Hardware/Software co-programmable framework for computational SSDs to accelerate deep learning service on large-scale graphs," in *Proc. 20th USENIX Conf. File Storage Technol.*, 2022, pp. 147–164. [Online]. Available: <https://www.usenix.org/conference/fast22/presentation/kwon>



Yunho Jin received the master's degree in computer science and engineering from Seoul National University, Korea. He is currently working toward the PhD degree in electrical engineering with Harvard University. His research areas include specialized system for emerging applications and optimized memory systems.



Shine Kim received the PhD degree in computer science and engineering from Seoul National University, Korea. He is a principal software engineer with Samsung Electronics. His research areas include computer architecture and storage systems.



Tae Jun Ham received the PhD degree in electrical engineering from Princeton University. He is a senior software engineer with Google. Before joining Google, he was a postdoctoral researcher with Seoul National University. His main research area is hardware-software co-design for emerging applications and performance optimization for large-scale systems.



Jae W. Lee (Senior Member, IEEE) is a professor with the Department of Computer Science and Engineering, Seoul National University (SNU), Korea. His research areas include computer architecture and systems, parallel programming, and hardware security. His work has been recognized with various awards and honors, including SNU Sinyang Outstanding Young Professor Award (2021), IEEE Symposium on VLSI Circuits Most Frequently Cited Paper Award (2017), ACM ASPLOS Most Influential Paper Award (2014), and IEEE Micro "Top Picks" (2020).

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.